

vmsOpenAcars — Documentación de Arquitectura

Versión del documento: 0.4.6

Última actualización: 2026-05-08

¿Qué es vmsOpenAcars?

Cliente ACARS de escritorio para Windows que conecta un simulador de vuelo con una aerolínea virtual basada en **phpVMS v7**. El piloto vuela en el simulador y la aplicación registra, valida y envía el PIREP automáticamente, sin intervención manual.

Stack Técnico

Componente	Tecnología
Plataforma	.NET Framework 4.8, C# 7.3
UI	Windows Forms (WinForms)
Sim → App	FSUIPC / FSUIPC7 — FSUIPCClientDLL 3.3.16 · XUIPC (X-Plane)
App → VA	phpVMS v7 REST API
Serialización	Newtonsoft.Json 13
Plan de vuelo	SimBrief API (JSON)
BD de pista	System.Data.SQLite 1.0.119 (LittleNavMap)
BD de historial	System.Data.SQLite 1.0.119 (landing_log.sqlite)
Gráficos	System.Windows.Forms.DataVisualization (incluido en .NET 4.8)
PDF	PdfiumViewer 2.13 + pdfium.dll x64
Clima / QNH	METAR vía WeatherService
Localización	JSON (en.json / es.json)
Configuración	App.config + Settings.settings

Estructura de Carpetas

vmsOpenAcars/

└─ Controls/

monitor)

└─ Core/

└─ Flight/

FlightManager, FlightTimer

└─ Helpers/

AppInfo

└─ Db/

RunwayService (LittleNavMap SQLite)

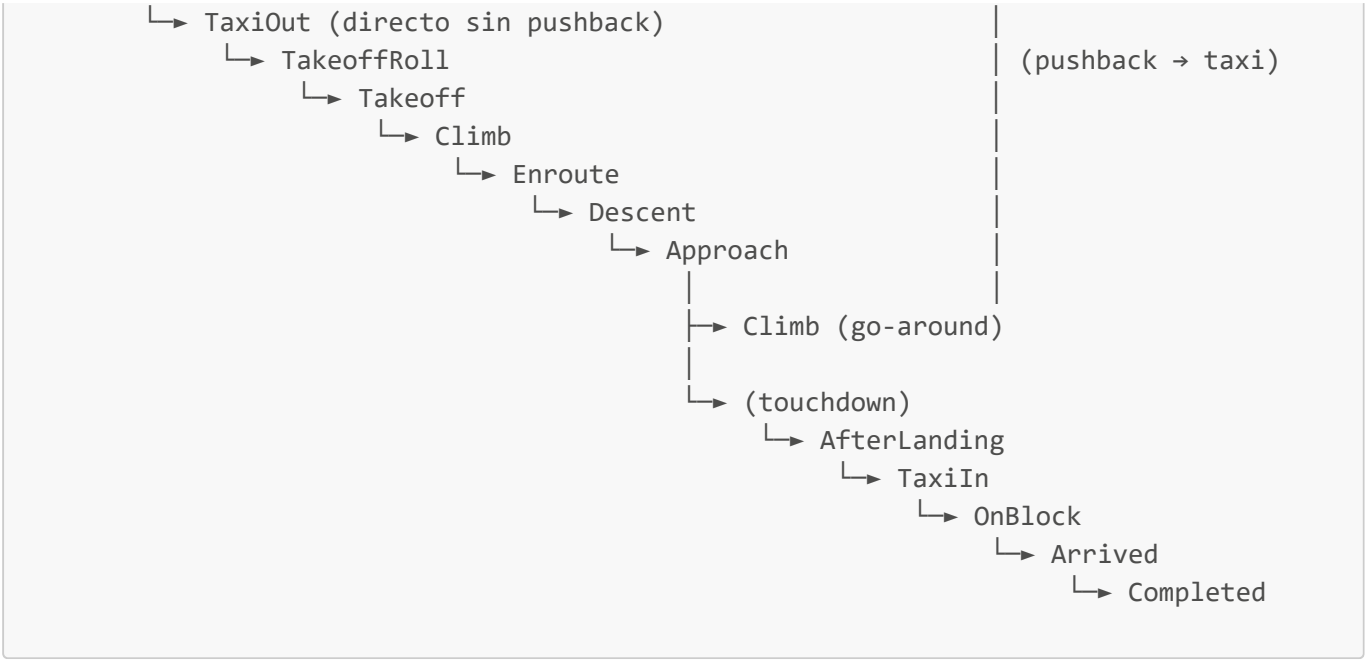
1 / 18

Arquitectura General

La comunicación ViewModel → Vista se hace mediante **eventos** (`Action<T>`, `Func<T>`). La Vista nunca llama directamente a servicios de infraestructura.

Fases de Vuelo

```
graph LR
    Idle --> Boarding
    Boarding --> Pushback
    Pushback --> Idle
```



Cada transición envía un código de estado al servidor phpVMS:

Fase	Código phpVMS
Boarding	BST
Pushback	PBT
TaxiOut / TaxiIn	TXI
Takeoff	TOF
Climb	ICL
Enroute	ENR
Descent	APR
Approach	FIN
Landing	LDG
OnBlock / Completed	ARR

Módulos

FsuipcService

Servicio de lectura del simulador. Opera en un timer a **50 ms** e interpola los offsets FSUIPC.

Offsets principales leídos:

Dato	Offset	Conversión
Latitud	0x0560	$raw \times 90 / (10001750 \times 65536^2)$
Longitud	0x0568	$raw \times 360 / 2^{64}$

Dato	Offset	Conversión
Altitud MSL	0x0570	$(raw / 65536^2) \times 3.28084 \text{ ft}$
Heading	0x0580	$raw \times 360 / 2^{32}$
Ground Speed	0x02B4	$(raw / 65536) \times 1.94384 \text{ kts}$
Vertical Speed	0x02C8	$(raw / 256) \times 196.85 \text{ fpm}$
IAS	0x02BC	$raw / 128 \text{ kts}$
Radar Altimetro	0x31E4	metros → feet
Luces	0x0D0C	bitfield (ver tabla de luces)
QNH (Kohlsman)	0x0330	$raw / 16.0 \text{ hPa}$
Viento velocidad	0x0E90	kts directo
Viento dirección	0x0E92	$raw \times 360 / 65536 \text{ grados}$
OAT	0x0E8C	$raw / 256 \text{ °C}$
G-Force	0x11BA	$raw / 625.0$

Bitfield de luces (0x0D0C):

Bit	Luz
0	NAV
1	BEACON
2	LANDING
3	TAXI
4	STROBE
5	SEAT BELT SIGN

Detección de eventos con debounce:

Evento	Debounce
NAV light	1.5 s
STROBE light	1.5 s
BEACON light	1.5 s
LANDING light	1.5 s
TAXI light	1.5 s
Parking Brake	2.0 s
Flaps	500 ms + histéresis 1%

Evento	Debounce
Touchdown / Liftoff	2.0 s + umbral GS

Telemetría adaptativa por fase:

Fase	Intervalo
Taxi	30 s
Takeoff / Approach	5 s
Climb / Descent	15 s
Enroute	30 s
Otros	30 s

FlightManager

Núcleo de la lógica del vuelo. Recibe `RawTelemetryData` cada ciclo y:

1. Actualiza propiedades públicas (altitud, GS, VS, luces, motores...)
2. Aplica **debounce de luces** (2 s) antes de actualizar los campos internos usados en compliance
3. Detecta transiciones de fase con umbrales relativos al plan de vuelo
4. Llama a `CheckProcedureAtPhaseEntry()` en cada transición
5. Llama a `CheckViolations()` cada ciclo mientras el avión está airborne y hay PIREP activo

Propiedades públicas de touchdown (expuestas para LandingLogService):

```
public double TouchdownDistanceFt    => _touchdownDistanceFt;
public double TouchdownCenterlineFt => _touchdownCenterlineDeviationFt;
public string TouchdownRunwayName    => _touchdownRunwayName;
public double TouchdownGForce        => _touchdownGForce;
```

Cálculo de AGL

Condición	Fuente
En tierra (<code>IsOnGround = true</code>)	0 siempre
Fase Enroute	Radar altímetro (<code>RadarAltitude</code>); fallback a baro si = 0
Resto de fases aéreas	<code>CurrentAltitude - ReferenceAirportElevation</code>

`ReferenceAirportElevation` devuelve la elevación del aeropuerto de **salida** en fases de despegue/climb, y la elevación del aeropuerto de **destino** en fases de llegada.

El radar altímetro **no se usa en aproximación** porque en terrenos montañosos (Andes, Alpes, Himalaya) lee la orografía bajo las ruedas, que puede estar muy por encima del aeropuerto, dando un AGL incorrecto.

Detección de Go-Around

Un go-around se confirma si se cumplen **todos** los criterios simultáneamente:

Criterio	Valor
VS positivo	> 600 fpm
Duración sostenida	≥ 8 segundos consecutivos
Tiempo mínimo en fase Approach	≥ 30 segundos
AGL (MSL – destElev)	entre 100 y 3 000 ft

El umbral de 600 fpm elimina falsos positivos por turbulencia o ajustes de pitch en final.

Compliance de Procedimientos (penalizaciones)

Momento	Verificación	Penalización
Motores ON	BEACON encendida	–5 pts
Pushback	NAV encendida	–5 pts
TaxiOut	NAV + TAXI encendidas	–5 pts c/u
TakeoffRoll	STROBE + LANDING encendidas	–5 pts c/u
Vuelo < 10 000 ft	LANDING encendida	–5 pts
TakeoffRoll	QNH ±2 hPa vs METAR origen	–5 pts
Gate 1 000 ft AGL (Approach)	QNH ±2 hPa vs METAR destino	–5 pts

RunwayService — [Db/RunwayService.cs](#)

Consulta la base de datos SQLite de **LittleNavMap** para cálculos precisos de posición en pista. Ruta configurada en [App.config](#) clave `lnm_db_path`.

API pública:

```
bool IsAvailable
RunwayTouchdownResult FindTouchdownRunway(airport, lat, lon, heading) //
touchdown zone + centreline
RunwayTouchdownResult FindTakeoffRunway(airport, lat, lon, heading) // pista de
despegue
RunwayEntry FindRunwayEntry(airport, lat, lon, heading) // entrada
a pista
string FindNearestTaxiway(airport, lat, lon) // taxiway
más cercano
HoldingPoint FindHoldingPoint(airport, lat, lon, heading) // holding
short
ParkingSpot FindNearestParking(airport, lat, lon) // gate /
parking
```

```

RunwayTouchdownResult GetRunwayThreshold(airport, lat, lon, heading) // umbral
para captura de aproximación – lat/lon desambiguan pistas paralelas
(double DistNm, double LateralFt) ComputeApproachMetrics(...) //
proyección flat-earth (static)
// ILS / Approach (v0.4.4)
ILsData GetILsForRunway(airport, runwayName) //
frecuencia MHz, curso ILS, gs_pitch; null si no existe o no es ILS
ApproachInfo GetApproachType(airport, runwayName) // mejor
procedimiento (ILS > RNAV > otro)
IList<ApproachFix> GetApproachFixes(approachId) // fixes
IF/FAF/MAP del procedimiento

```

`RunwayTouchdownResult` incluye: `ThresholdDistanceFt`, `CenterlineDeviationFt`, `RunwayName`, `ThresholdLat`, `ThresholdLon`, `ThresholdHeading`.

ILS / Approach result types (v0.4.4):

- `ILsData` — `FrequencyMhz`, `Course`, `GlideSlopePitch`, `RunwayName`, `ThresholdLat/Lon/ElevFt`
- `ApproachInfo` — `ApproachId`, `Type` ("ILS", "RNAV"...), `RunwayName`, `HasVerticalGuidance`
- `ApproachFix` — `Name`, `FixType` ("IF", "FAF", "MAP"), `Lat`, `Lon`, `AltitudeFt`

Esquema de BD LittleNavMap (verificado en producción):

```

airport    → airport_id, ident, lonx, laty
runway     → runway_id, airport_id, primary_end_id, secondary_end_id, width (ft),
length (ft)
runway_end → runway_end_id, name, heading, lonx, laty, offset_threshold
taxi_path  → taxi_path_id, airport_id, type ('T'=taxiway, 'P'=pavement), name,
start_lonx/laty, end_lonx/laty
parking    → parking_id, airport_id, type, name, number, suffix, radius, lonx,
laty

```

Geometría flat-earth:

```

dN = (lat - thLat) × 111320
dE = (lon - thLon) × 111320 × cos(thLat_rad)
along = dE × sin(hdg_rad) + dN × cos(hdg_rad) → distancia al umbral (m)
cross = dE × cos(hdg_rad) - dN × sin(hdg_rad) → desviación centreline (m,
signed)

```

Radios de búsqueda:

Elemento	Radio
Runway footprint	1.5× semi-ancho real + 30 m buffer longitudinal
HoldingPoint	200 m del umbral

Elemento	Radio
Taxiway	300 m del segmento más cercano
Parking	200 m

ScoringService

Calcula un score de 0–100 al finalizar el vuelo. El score comienza en 100 y se aplican deducciones:

Criterio	Máx. deducción	Escala
Landing Rate	–40 pts	≤100 fpm: 0 / ≤200: –5 / ≤300: –15 / ≤400: –25 / ≤600: –35 / >600: –40
G-Force touchdown	–15 pts	≤1.3g: 0 / ≤1.5g: –7 / >1.5g: –15
Bank Angle touchdown	–10 pts	≤2°: 0 / ≤5°: –5 / >5°: –10
Pitch Angle touchdown	–10 pts	1°–5°: 0 (ideal) / fuera de rango: –5 a –10
Overspeed	–15 pts	0 eventos: 0 / 1: –7 / ≥2: –15
Lights Compliance	–10 pts	–5 pts por violación, cap –10
Stabilized Approach (1000 ft)	–15 pts	Evalúa speed, VS, bank, pitch, gear y flaps a 1000 ft AGL
QNH Compliance	–10 pts	–5 pts si Δ > 2 hPa — salida (TakeoffRoll) + llegada (gate 1000 ft AGL), independientes
IVAO Offline	–5 pts	–5 si el vuelo se realizó sin conexión IVAO
On-Time Departure	–5 pts	–5 si Blocks Off difiere > 10 min del STD (sched_out)
Touchdown Zone	–7 pts	≤1500 ft = 0 / ≤2500 ft = –3 / >2500 ft = –7 · requiere LNM DB
Centreline Deviation	–7 pts	≤10 ft = 0 / ≤30 ft = –3 / >30 ft = –7 · requiere LNM DB
Localizer Alignment	–5 pts	ILS not tuned –3; heading >5° x2 max –2; cap –5 · requiere LNM DB + ILS approach
Minimums Compliance	–5 pts	–5 si descenso bajo DA (threshold elevation + 200 ft) sin aterrizar

Los criterios **Touchdown Zone** y **Centreline Deviation** solo se evalúan si TouchdownDistanceFt > 0 (es decir, si RunwayService pudo consultar la BD de LittleNavMap). Los criterios **Localizer Alignment** y **Minimums Compliance** solo se evalúan si se detectó un procedimiento ILS para la pista de aterrizaje.

Landing ratings: Butter (≤100 fpm) · Smooth · Normal · Hard · Very Hard · Slam (≥600 fpm)

LandingLogService — `Services/LandingLogService.cs`

Gestiona la base de datos SQLite local `landing_log.sqlite`. Ruta configurada en `App.config` clave `landing_log_path`.

Tablas:

```
flights (
    id INTEGER PRIMARY KEY,
    flight_number, origin, destination, runway_name, metar_raw TEXT,
    flight_date TEXT, -- ISO 8601 UTC
    landing_rate_fpm INTEGER,
    score INTEGER,
    g_force REAL,
    touchdown_dist_ft REAL,
    centerline_dev_ft REAL
)

approach_track (
    id INTEGER PRIMARY KEY,
    flight_id INTEGER, -- FK → flights.id
    seq_no INTEGER,
    lat REAL, lon REAL,
    alt_ft REAL, agl_ft REAL,
    ias_kt REAL, vs_fpm REAL,
    heading_deg REAL,
    dist_nm REAL, -- distancia al umbral (positiva = antes del umbral)
    lateral_ft REAL -- desviación centreline (signed: + derecha, - izquierda)
)
```

API pública:

```
bool IsAvailable
int SaveFlight(FlightRecord record, IList<ApproachTrackPoint> track)
void DeleteFlight(int id) // borra en transacción: approach_track primero,
    luego flights
List<FlightRecord> GetFlights()
List<ApproachTrackPoint> GetTrackPoints(int flightId)
bool HasFlights()
void SeedMockData() // solo disponible en #if DEBUG — 5 vuelos SKRG
    RWY 01
```

Flujo de captura de aproximación:

```
Phase → Approach
    → RunwayService.GetRunwayThreshold(dest, lat, lon, heading) →
    _approachThreshold
    → Task.Run(LoadApproachData) //
```

```

v0.4.4
    → GetIlsForRunway() + GetApproachType() + GetApproachFixes()
    → _flightManager.SetApproachData(ils, approach, fixes)
    → _approachBuffer.Clear()
OnRawDataUpdated (cada 50 ms)
    → si phase=Approach && AGL < 3000 ft && ≥ 2 s desde último punto
    → ComputeApproachMetrics(threshold, lat, lon) → (distNm, lateralFt)
    → _approachBuffer.Add(ApproachTrackPoint)
SendPirep()
    → SnapshotLandingRecord() ← captura plan + touchdown ANTES de
FilePirep
    → FilePirep() → ResetFlightState() ← borra _activePlan y touchdown data
    → éxito → SaveLandingRecord(record)
        → record.Score = LastFlightScore ← no se resetea en ResetFlightState
        → LandingLogService.SaveFlight(record, _approachBuffer)
        → _approachBuffer.Clear()

```

`SnapshotLandingRecord()` debe ejecutarse **antes** de awaitar `FilePirep()`. `LastFlightScore` es la única propiedad que `ResetFlightState()` no borra, por lo que puede leerse después.

LandingAnalysisForm — `UI/Forms/LandingAnalysisForm.cs`

Ventana no-modal que muestra 4 gráficos de la trayectoria de aproximación. Soporta modo individual y modo comparación (2-5 vuelos).

Constructor: `LandingAnalysisForm(IList<FlightRecord Record, List<ApproachTrackPoint> Track> flights)`

Gráficos:

Gráfico	Y	Referencia
Vertical Profile	AGL (ft)	Línea 3° = dist_nm × 319 ft/NM
Lateral Deviation	Desviación (ft, signed ±)	Línea cero = eje de pista
IAS	Velocidad indicada (kt)	Línea promedio ≈ Vref
VS	Vertical speed (fpm)	Línea cero

- Eje X invertido: 5 NM izquierda → 0 (umbral) derecha
- Suavizado Gaussiano (window=7, σ =window/4) aplicado a Lateral, IAS, VS — NO a Vertical Profile
- En comparación, cada vuelo usa un color de la paleta `TrackColors[]` (azul, naranja, verde, violeta, dorado)
- Nombres de series: "{FlightNumber} #{i+1}" en comparación; "Actual" en modo individual

OsdOverlayForm — `UI/Forms/OsdOverlayForm.cs`

Ventana de notificaciones en pantalla (OSD — On-Screen Display). TopMost, sin borde, sin entrada en la barra de tareas, completamente click-through (`WM_NCHITTEST` → `HTTRANSPARENT`).

Enum de severidad:

```
public enum OsdSeverity { Info, Success, Warning, Critical }
```

Colores de texto por severidad:

Severidad	Color
Info	Azul claro #A0DCFF
Success	Lima #64FF82
Warning	Dorado #FFD700
Critical	Rojo #FF6E6E

Estados de animación:

```
Idle → FadeIn (Opacity 0→target, paso 0.06/tick) → Hold (_holdTicks) → FadeOut  
(paso -0.04/tick) → Idle+Hide
```

Para **Critical**: en lugar de FadeIn, activa el `_flashTimer` (220 ms) que alterna `BgFlashOn/BgFlashOff` durante 3 ciclos completos, luego transiciona a Hold → FadeOut.

API pública:

```
void ShowMessage(string text, OsdSeverity severity, int durationMs = 4000)  
void HideOsd()
```

`ShowMessage()` es thread-safe (usa `InvokeRequired`). Recalcula la posición en pantalla en cada llamada usando `Screen.Bounds` (área completa, incluyendo zona de taskbar) para funcionar correctamente tanto en modo ventana como en fullscreen.

Posicionamiento: centrado horizontalmente, 40 px desde el borde superior de la pantalla configurada (`osd_screen_index`). Si el índice está fuera de rango, usa la pantalla primaria.

Puntos de disparo en MainViewModel:

Evento	Mensaje	Severidad
<code>StartFlight()</code> confirmado	ACARS ACTIVE	Success
<code>OnFlightPhaseChanged(TaxiOut)</code>	TAXI OUT	Info
<code>OnFlightPhaseChanged(TakeoffRoll)</code>	TAKEOFF ROLL	Info
<code>OnFlightPhaseChanged(Enroute)</code>	CRUISE	Info

Evento	Mensaje	Severidad
OnFlightPhaseChanged(Descent)	DESCENDING	Info
OnFlightPhaseChanged(Approach)	APPROACH	Info
OnFlightPhaseChanged(OnBlock)	ON BLOCK	Info
Touchdown detectado	<calificación> -XXX fpm X.Xg	varía por fpm
Touch-and-go	TOUCH AND GO	Warning
SendPirep() exitoso	PIREP FILED – SCORE: XX/100	Success

Integración en MainForm:

```
_viewModel.OnOsdMessage += (text, severity) =>
    _osd.ShowMessage(text, severity, AppConfig.OsdDurationMs);
```

AircraftPerformanceTable y Detección de Overspeed

¿Qué es Vmo?

Vmo (Velocity Maximum Operating) es la velocidad máxima operativa publicada en el FCOM/AFM de cada aeronave, expresada en nudos IAS. En vmsOpenAcars es el único umbral de velocidad para penalizar al piloto (no se implementa Mmo por requerir datos de temperatura y presión variable con la altitud).

Resolución del Tipo de Aeronave

1. Match exacto (case-insensitive) "B738" → Boeing 737-800, Vmo 340 kts
↓ no encontrado
2. Prefijo de 4 chars
↓ no encontrado
3. Prefijo de 3 chars "B38M" → prefijo "B3" → familia B737 MAX, Vmo 340 kts
↓ no encontrado
4. Default genérico 320 kts

Tabla de Vmo por Categoría

Pistones ligeros y GA

Tipo ICAO	Aeronave	Vmo (kts)
C172	Cessna 172	163
C182	Cessna 182	175

Tipo ICAO	Aeronave	Vmo (kts)
C208	Cessna Caravan	175
PA28	Piper PA-28	148
PA44	Piper Seminole	169
BE58	Beechcraft Baron 58	195
BE20	King Air 200	260
BE30	King Air 300/350	260
PC12	Pilatus PC-12	210

Turboprops regionales

Tipo ICAO	Aeronave	Vmo (kts)
AT42–AT46	ATR 42 (todas variantes)	250
AT72–AT76	ATR 72 (todas variantes)	250
DH8A / DH8B	Dash 8-100/200	220
DH8C / DH8D	Dash 8-300/400 (Q400)	260
SB20	Saab 2000	290
SF34	Saab 340	250
E120	Embraer 120	255

Jets regionales

Tipo ICAO	Aeronave	Vmo (kts)
CRJ2	CRJ-200	320
CRJ7	CRJ-700	320
CRJ9	CRJ-900	320
CRJX	CRJ-1000	320
E135 / E145	ERJ-135 / ERJ-145	320
E170 / E175	Embraer E170 / E175	320
E190 / E195	Embraer E190 / E195	320

Narrow-body jets

Tipo ICAO	Aeronave	Vmo (kts)
A318–A321	Airbus A318/319/320/321	350

Tipo ICAO	Aeronave	Vmo (kts)
A20N / A21N	A320neo / A321neo	350
B731–B739	Boeing 737 Clásico y NG	340
B37M–B3XM	Boeing 737 MAX 7/8/9/10	340
MD82 / MD83	McDonnell Douglas MD-80	340

Wide-body jets

Tipo ICAO	Aeronave	Vmo (kts)
A332 / A333 / A339	Airbus A330-200/300/900neo	330
A342–A346	Airbus A340 (todas variantes)	330
A359 / A35K	Airbus A350-900/1000	330
A388	Airbus A380-800	330
B752 / B753	Boeing 757-200/300	350
B762–B764	Boeing 767-200/300/400	350
B772–B77W	Boeing 777-200/300/ER/LR	330
B779	Boeing 777X	330
B787–B78X	Boeing 787-8/9/10	330
B744 / B748	Boeing 747-400/8	365
B74F / B74S	Boeing 747-400F / 747SP	365

Prefijos fallback (3 chars):

Prefijo	Familia	Vmo (kts)
A32	A320 family	350
A33	A330 family	330
A34	A340 family	330
A35	A350 family	330
A38	A380 family	330
B73	B737 family	340
B3	B737 MAX	340
B74	B747 family	365
B75	B757 family	350
B76	B767 family	350

Prefijo	Familia	Vmo (kts)
B77	B777 family	330
B78	B787 family	330
AT4	ATR 42 family	250
AT7	ATR 72 family	250
DH8	Dash 8 family	260
DHC	DHC family	215
CRJ	CRJ family	320
E17	E-jet 170	320
E19	E-jet 190	320
BE2	King Air	260
BE3	King Air 350	260
C20	Caravan	175

Si ningún prefijo coincide → **320 kts** (default genérico conservador).

Ciclo de Detección de Overspeed

`CheckViolations()` se llama **una vez por ciclo de telemetría** (~50 ms) mientras airborne con PIREP activo:

```

Por cada ciclo:
  IAS actual > Vmo?
    SÍ y _wasOverspeed=false → _overspeedCount++ + log
    NO                        → _wasOverspeed=false

```

El flag `_wasOverspeed` actúa como latch: un overspeed sostenido cuenta como **un solo evento**.

Eventos registrados	Deducción
0	0 pts
1	-7 pts
≥ 2	-15 pts (máximo)

SimbriefEnhancedService

`GenerateDispatchUrl()` — Genera URL para pre-cargar el plan en SimBrief:

- Hora de salida: UTC actual + 30 min
- Parámetros: aerolínea, número, origen, destino, tipo aeronave, matrícula, ruta, CI, unidades

FetchAndParseOFP() — Descarga y parsea el JSON de la API de SimBrief:

- Construye **SimbriefPlan** completo: routing, combustible, pesos, tiempos, elevaciones, URL del PDF
- Maneja que **files.pdf** puede ser string o objeto **{name, link}**
- Campos de combustible: **BlockFuel** ← **fuel.plan_ramp** · **TripFuel** ← **fuel.enroute_burn**

OFP PDF — Flujo Completo

```

FlightPlannerForm (acepta OFP)
├─ SetActivePlan(plan)
└─ DownloadOFPPdfAsync() ← background, fire-and-forget
    └─ plan.LocalPdfPath = "/temp/vmsOFP_xxxx.pdf"

BtnOfp_Click
├─ GetCachedOFPPath() ← archivo en disco?
│   ├── Sí → OFPViewerForm(cachedPath) [instantáneo]
│   └─ No → DownloadOFPPdfAsync() → OFPViewerForm(newPath)
└─ OFPViewerForm.OnFormClosed → File.Delete(tempPath)
  
```

Modelos Principales

SimbriefPlan

```

// Vuelo
string FlightNumber, Airline, Origin, Destination, Alternate, Route
int DestinationElevation, OriginElevation, CruiseAltitude, EstTimeEnroute
long TimeGenerated, ScheduledOffTime

// Aeronave
string Aircraft, AircraftIcao, Registration, FlightId, BidId

// Combustible / Pesos
double BlockFuel, TripFuel, DepartureFuel, PayLoad, ZeroFuelWeight
string Units // "KG" | "LBS"
int PaxCount

// PDF
string PdfUrl, LocalPdfPath
  
```

FlightRecord


```

int    Id, LandingRateFpm, Score
string FlightNumber, Origin, Destination, RunwayName, MetarRaw
DateTime FlightDate
double GForce, TouchdownDistFt, CenterlineDevFt
// Display helpers:
string DisplayDate  → "yyyy-MM-dd HH:mm"
string DisplayRoute → "ORIG → DEST"
string DisplayScore → "score/100"

```

ApproachTrackPoint

```

int    FlightId, SeqNo
double Lat, Lon, AltFt, AglFt, IasKt, VsFpm, HeadingDeg, DistNm, LateralFt

```

FlightPhase (enum)

Idle → Boarding → Pushback → TaxiOut → TakeoffRoll → Takeoff → Climb → Enroute → Descent
 → Approach → Landing → Landed → AfterLanding → TaxiIn → OnBlock → Arrived → Completed

Localización

Dos archivos JSON en `Languages/`: `en.json` y `es.json`.

Acceso mediante el helper estático `L_("key")` importado con `using static vmsOpenAcars.Helpers.L`.

El idioma se selecciona en `SettingsForm` y se persiste en `App.config`.

Configuración (App.config / AppConfig)

Clave	Default	Descripción
<code>polling_interval_ms</code>	50	Intervalo de polling FSUIPC
<code>update_interval_taxi</code>	30	Telemetría en taxi (s)
<code>update_interval_takeoff</code>	5	Telemetría en despegue (s)
<code>update_interval_climb</code>	15	Telemetría en subida (s)
<code>update_interval_cruise</code>	30	Telemetría en crucero (s)
<code>update_interval_descent</code>	15	Telemetría en descenso (s)
<code>update_interval_approach</code>	5	Telemetría en approach (s)
<code>fuel_tolerance_percent</code>	10	Tolerancia combustible (%)
<code>fuel_tolerance_absolute</code>	50	Tolerancia combustible (kg)
<code>simbrief_civalue</code>	30	Cost Index para SimBrief

Clave	Default	Descripción
<code>simbrief_units</code>	lbs	Unidades combustible SimBrief
<code>lnm_db_path</code>	(vacío)	Ruta al <code>airports.sqlite</code> de LittleNavMap
<code>landing_log_path</code>	(vacío)	Ruta al archivo <code>landing_log.sqlite</code>
<code>osd_enabled</code>	true	Activa el overlay OSD
<code>osd_duration_seconds</code>	4	Tiempo de visualización por notificación (s)
<code>osd_screen_index</code>	0	Índice de pantalla para el OSD (0 = primaria)
<code>osd_opacity</code>	90	Opacidad del OSD (10–100 %)

Requisitos de Instalación

1. **Simulador**: ver tabla de compatibilidad

Simulador	Plugin requerido
MSFS 2020 / 2024	FSUIPC 7
Prepar3D v5 / v6	FSUIPC 6
Prepar3D v4	FSUIPC 5 o 6
Prepar3D v1 / v2 / v3	FSUIPC 4
FSX / FSX: Steam Edition	FSUIPC 4
X-Plane 11 / 12	XUIPC (plugin para X-Plane)

2. **pdfium.dll** (x64): incluido en el build (`PdfiumViewer.Native.x86_64.no_v8-no_xfa`)
3. **Plataforma destino**: x64 (`Prefer32Bit=false` en el proyecto)
4. **.NET Framework 4.8** en el sistema

Notas de Build

- Configuración **Release**: `<AppConfig></AppConfig>` en el PropertyGroup de Release evita que MSBuild copie `App.config` sobre `vmsOpenAcars.exe.config`. El `App.Release.config` de producción en `bin\Release\` queda intacto.
- `pdfium.dll` se copia siempre al directorio de salida (`CopyToOutputDirectory=Always`)
- `Languages/*.json` se copian con `PreserveNewest`
- `SeedMockData()` en `LandingLogService` solo compila en configuración **Debug** (`#if DEBUG`)